SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - Database Design and Connectivity

Course Code:	DSA0501	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	AT1 – Database Design and Connectivity
Weightage:		**Total Marks:**	20 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Student Name:	SURYA JK	**Reg. No.:**	192324285
Section / Batch:	A/2023	**Date of Submission:**	29/07/2026
Faculty Incharge:	K Veena devi	**Project Mode:**	Individual Submission

Code of Conduct

I _Surya JK/192324285_ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____SURYA JK_____

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Sample Answers – Selected Questions

Question 1: Student Database Design and Connectivity (10 Marks)

1. Requirement Analysis

- The college needs a centralized system to store and manage student records.
- Each student must be uniquely identified using a Student ID (Primary Key).
- The system should store Name, Age, Department, Email, and Phone Number for every student.
- The application must be able to create the table automatically if it does not already exist, insert new student records, and display all stored records on demand.

2. ER Diagram Design

A single entity 'Student' is sufficient since all attributes describe one real-world object with no repeating groups, so no relationship/second entity is required.

Entity: STUDENT	
StudentID (PK)	INTEGER
Name	TEXT
Age	INTEGER
Department	TEXT
Email	TEXT
Phone	TEXT

3. Table Design & Normalization

```
CREATE TABLE IF NOT EXISTS Student (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Age INTEGER,
    Department TEXT,
    Email TEXT UNIQUE,
    Phone TEXT
);
```

- 1NF: Every column holds a single atomic value; there are no repeating groups.
- 2NF: The table has a single-column primary key (StudentID), so there is no partial dependency.
- 3NF: All non-key columns (Name, Age, Department, Email, Phone) depend only on StudentID and not on each other, so there is no transitive dependency.

4. Database Connectivity using Python

```
import sqlite3

# Connect to (or create) the SQLite database
conn = sqlite3.connect('student.db')
cursor = conn.cursor()

# Create the Student table if it does not exist
cursor.execute('''
CREATE TABLE IF NOT EXISTS Student (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Age INTEGER,
    Department TEXT,
    Email TEXT UNIQUE,
    Phone TEXT
```

```
)
''')
conn.commit()
```

5. CRUD Operations

- Create: insert at least five student records.
- Read: display all records stored in the table.
- Update/Delete are also included below to demonstrate the complete CRUD cycle.

```
students = [
    ('Arun Kumar', 20, 'CSE', 'arun@mail.com', '9876543210'),
    ('Divya Sri', 19, 'IT', 'divya@mail.com', '9876543211'),
    ('Karthik R', 21, 'ECE', 'karthik@mail.com', '9876543212'),
    ('Priya M', 20, 'CSE', 'priya@mail.com', '9876543213'),
    ('Sanjay V', 22, 'MECH', 'sanjay@mail.com', '9876543214'),
]

# CREATE - insert records
cursor.executemany('''
INSERT INTO Student (Name, Age, Department, Email, Phone)
VALUES (?, ?, ?, ?, ?)
''', students)
conn.commit()

# READ - display all records
cursor.execute('SELECT * FROM Student')
for row in cursor.fetchall():
    print(row)

# UPDATE - change a student's phone number
cursor.execute('UPDATE Student SET Phone = ? WHERE Name = ?', ('9998887770', 'Arun Kumar'))
conn.commit()

# DELETE - remove a student record
cursor.execute('DELETE FROM Student WHERE Name = ?', ('Sanjay V',))
conn.commit()

conn.close()
```

Question 3: Employee and Department Database Design (10 Marks)

1. Requirement Analysis

- The company needs to store Department details and Employee details separately to avoid data redundancy.
- Every department must be uniquely identified, and every employee must belong to exactly one department.
- The system must support creating both tables, inserting sample data, and retrieving each employee's name together with their department name using a JOIN.

2. ER Diagram Design

Two entities, Department and Employee, are linked by a one-to-many relationship: one Department can have many Employees, but each Employee belongs to only one Department. DeptID is the Primary Key of Department and a Foreign Key in Employee.

Entity: DEPARTMENT (1) —— (M) EMPLOYEE	
DeptID (PK)	INTEGER
DeptName	TEXT
EmpID (PK)	INTEGER
EmpName	TEXT
Salary	REAL
DeptID (FK -> Department.DeptID)	INTEGER

3. Table Design & Normalization

```
CREATE TABLE IF NOT EXISTS Department (
    DeptID INTEGER PRIMARY KEY,
    DeptName TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Employee (
    EmpID INTEGER PRIMARY KEY,
    EmpName TEXT NOT NULL,
    Salary REAL,
    DeptID INTEGER,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

- 1NF: All attributes in both tables are atomic (e.g. one DeptName per row).
- 2NF: Both tables use single-column primary keys, so there are no partial dependencies.
- 3NF: DeptName is stored only once in Department instead of being repeated in every Employee row, removing transitive dependency and redundancy.

4. Database Connectivity using Python

```
import sqlite3

conn = sqlite3.connect('company.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE IF NOT EXISTS Department (
    DeptID INTEGER PRIMARY KEY,
    DeptName TEXT NOT NULL
)
''')

cursor.execute('''
CREATE TABLE IF NOT EXISTS Employee (
```

```
    EmpID INTEGER PRIMARY KEY,
    EmpName TEXT NOT NULL,
    Salary REAL,
    DeptID INTEGER,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
)
''')
conn.commit()
```

5. CRUD Operations

- Create: insert sample departments and employees.
- Read: display employee name with department name using an SQL JOIN.
- Update/Delete shown below to complete the CRUD cycle.

```
departments = [(1, 'CSE'), (2, 'ECE'), (3, 'MECH')]
cursor.executemany('INSERT INTO Department VALUES (?, ?)', departments)

employees = [
    (101, 'Ravi Shankar', 55000, 1),
    (102, 'Meena Kumari', 48000, 2),
    (103, 'Suresh Babu', 52000, 1),
    (104, 'Anitha Raj', 47000, 3),
]
cursor.executemany('INSERT INTO Employee VALUES (?, ?, ?, ?)', employees)
conn.commit()

# READ with JOIN - employee name with department name
cursor.execute('''
SELECT Employee.EmpName, Department.DeptName
FROM Employee
JOIN Department ON Employee.DeptID = Department.DeptID
''')
for row in cursor.fetchall():
    print(row)

# UPDATE - give an employee a raise
cursor.execute('UPDATE Employee SET Salary = ? WHERE EmpID = ?', (60000, 101))
conn.commit()

# DELETE - remove an employee who has left
cursor.execute('DELETE FROM Employee WHERE EmpID = ?', (104,))
conn.commit()

conn.close()
```

Question 6: Banking Account Management System (10 Marks)

1. Requirement Analysis

- The bank needs to store each customer's account information securely and consistently.
- Each account must be uniquely identified using an Account Number (Primary Key).
- The system must support creating the account table, inserting customer details, updating the balance of a specific account, and displaying all customer records.

2. ER Diagram Design

A single entity 'Account' models the requirement, since account, customer identity, and balance information all describe one bank account record.

Entity: ACCOUNT	
AccountNo (PK)	INTEGER
CustomerName	TEXT
AccountType	TEXT
Balance	REAL
Phone	TEXT

3. Table Design & Normalization

```
CREATE TABLE IF NOT EXISTS Account (
    AccountNo INTEGER PRIMARY KEY,
    CustomerName TEXT NOT NULL,
    AccountType TEXT CHECK(AccountType IN ('Savings','Current')),
    Balance REAL NOT NULL DEFAULT 0,
    Phone TEXT
);
```

- 1NF: Every attribute (CustomerName, AccountType, Balance, Phone) is atomic.
- 2NF: The table uses a single-column primary key (AccountNo), so there is no partial dependency.
- 3NF: All non-key attributes depend directly on AccountNo, with no transitive dependency between them.

4. Database Connectivity using Python

```
import sqlite3

conn = sqlite3.connect('bank.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE IF NOT EXISTS Account (
    AccountNo INTEGER PRIMARY KEY,
    CustomerName TEXT NOT NULL,
    AccountType TEXT CHECK(AccountType IN ('Savings','Current')),
    Balance REAL NOT NULL DEFAULT 0,
    Phone TEXT
)
''')
conn.commit()
```

5. CRUD Operations

- Create: insert customer account details.
- Update: change the balance of a specified customer.
- Read: display all customer records.
- Delete shown below to complete the CRUD cycle (e.g. closing an account).

```
accounts = [
    (1001, 'Kavitha Rani', 'Savings', 25000.00, '9000011111'),
    (1002, 'Mohan Das', 'Current', 150000.00, '9000011112'),
    (1003, 'Sneha Reddy', 'Savings', 8000.00, '9000011113'),
]

# CREATE - insert account records
cursor.executemany('INSERT INTO Account VALUES (?, ?, ?, ?, ?)', accounts)
conn.commit()

# UPDATE - update balance of a specified customer (deposit example)
cursor.execute('UPDATE Account SET Balance = Balance + ? WHERE AccountNo = ?', (5000, 1003))
conn.commit()

# READ - display all customer records
cursor.execute('SELECT * FROM Account')
for row in cursor.fetchall():
    print(row)

# DELETE - close an account
cursor.execute('DELETE FROM Account WHERE AccountNo = ?', (1002,))
conn.commit()

conn.close()
```